

C73_A1

Changze Wu 1008837840

February 2025

1 Q1

Choose greedy strategy: select the interval with largest duration first. Instead of selecting the interval that ends earliest (which is the correct strategy), this approach picks the interval with the longest duration first and then removes all conflicting intervals.

For any n , we can construct an input where this greedy strategy fails. Consider the set of n intervals:

$$I_1 = (1, n+1), I_2 = (1, 2), I_3 = (2, 3), I_4 = (3, 4) \dots I_n = (n-1, n)$$

in words:

1. the first interval I_1 spans a long duration from 1 to $n+1$
2. The remaining $n-1$ intervals are small and sequentially placed without overlapping each other.

Proof this algo is wrong :

Solution from the incorrect greedy strategy:

1. Pick I_1 because it has the longest duration
2. remove all other intervals
3. output contains 1 interval

Optimal solution

1. Picks all $n-1$ short intervals since they do not overlap.
2. Output contains $n-1$ intervals.

Since $1 \neq n-1$ for $n \geq 2$, this proves that the greedy strategy of selecting the longest interval first fails to maximize the number of non-overlapping intervals for any n .

QED

2 Q2

Proof: We will formally prove that the greedy algorithm given in the problem always produces a vertex cover of size at most $2 \cdot OPT(G)$, where $OPT(G)$ is the size of the minimum vertex cover.

we consider a matching set which is a set of edges which no two edges share a common vertex.

In this case, we take M as the set of edges which were picked by algo in the loop. Since the algorithm always removes all edges incident to u and v after selecting an edge (u, v) , it follows that: 1) No two edges in M share a common vertex and 2) M is matching.

we assume $S_{OPT} \subseteq V$ be the optimal set of vertex. Since every edge in M must be covered in any valid vertex cover, S_{OPT} must include at least one vertex from each edge in M . In this way, we can conclude that $OPT(G) \geq |M|$.

According to the greedy algo (line 4) we adds both endpoints of every edge in M , so it selects exactly $2|M|$ vertices. Since we have $OPT(G) \geq |M|$. then we have $2 \cdot OPT(G) \geq 2|M| = |S|$.

which proves that the algorithm's output is at most twice the optimal solution.QED

3 Q3

take a $n = 3$ for example

$$M = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$$

According to greedy algo present in question. we will first choose $M[1][1]$ and then choose $M[2][2]$ and when check $M[3][3]$ it will return fails.

however, it is easily to find that there is a diagonal $M[1][3], M[2][2], M[3][1]$, which prove the algo fails. QED

4 Q4

according to the question, we could get $a = 12, b = 5, d = \frac{3}{2}, c = \log_5 12$. According to Master Theorem, we compare c and d . $c = \log_5 12 \approx 1.5439 > 1.5 = d$, apply MT, then we get $T(n) = 12 \cdot T(n/5) + O(n^{\frac{3}{2}})$ and $T(n) = O(n^{\log_5 12})$. Since $\log_5 12 = 1.5439 < \log_2 3 = 1.585$, we can conclude that the algo in question is faster than Karatsuba.

However, it is slower than best known algorithm ($O(n \cdot \log n)$) for this problem

5 Q5

Through def of FFT, we can get the formula:

$$T(n) = 2T(n/2) + O(n)$$

Base case : when $n = 2$, we only need to calculate two point DFT, thus the complexity is $O(1)$, which is $T(n) \leq Cn \log n$

I.H. assume for $n = 2^k$, we also hold $T(n) \leq Cn \log n$, which C is a constant. WTP, when $n' = 2n = 2^{k+1}$, we also hold $T(n') \leq Cn' \log n'$, which is $T(2n) \leq C2n \log 2n$.

According to the formula we have

$$T(2n) = 2T(n) + O(2n) \leq 2 \cdot Cn \log n + O(2n)$$

In the meanwhile, we can also transform the $C2n \log 2n$:

$$C2n \log 2n = 2Cn(\log 2 + \log n) = 2Cn + 2Cn \log n$$

for $O(2n)$ is linear and we assume C is enough big, we can get $2Cn \geq O(2n)$. According to that we get

$$T(2n) \leq 2 \cdot Cn \log n + O(2n) \leq 2Cn + 2Cn \log n = C2n \log 2n$$

as wanted.

QED

6 Q6

when we look at the Toeplitz matrix, we can find that there are total $2n - 1$ diagonals, and each diagonal should represent the same value. thus, we can decide that we can represent $n \times n$ Toeplitz matrix in $2n - 1$ numbers.

Thus, we can store this matrix in such $T = [a_0, a_1, \dots, a_{n-1}, a_n, \dots, a_{2n-2}]$. First, we defined two polynomial:

Polynomial $p(x)$: It represents the stored Toeplitz matrix values and its coefficients are $p(x) = a_0 + a_1x + a_2x^2 + \dots + a_{2n-2}x^{2n-2}$

Polynomial $q(x)$: It represents the input vector v , zero-padded to size $2n-1$ and its coefficients are $q(x) = v_0 + v_1x + v_2x^2 + \dots + v_{n-1}x^{n-1} + 0 \cdot x^n + \dots + 0 \cdot x^{2n-2}$

The matrix-vector multiplication $A \cdot v$ corresponds to computing the first n elements of the polynomial convolution $p(x) \cdot q(x)$. Then using FFT, we can compute this convolution in $O(n \log n)$.

Input: $T[0 \dots 2n - 2]$ which is Toeplitz matrix representation and $v[0 \dots n - 1]$ is input vector. The algo will output $y[0 \dots n - 1]$

```

Algo(T, v):
1. p = [T[0], T[1], ..., T[2n-2]]    (length 2n-1)
2. q = [v[0], v[1], ..., v[n-1], 0, ..., 0]    (length 2n-1)
3. FFT_p = FFT(p)    // Transform p(x) to frequency domain
4. FFT_q = FFT(q)    // Transform q(x) to frequency domain
   // Convolution in frequency domain
5. FFT_result = FFT_p * FFT_q
6. result = Inverse_FFT(FFT_result)
7. y = [result[0], result[1], ..., result[n-1]]
8. Return y

```

In this algo, we construct two polynomial (line 1 & 2), cost $O(n)$. then in line 3 & 4 we compute FFT for two polynomial and cost $O(n \log n)$. then in line 5, we point-wise multiplication in frequency domain will cost $O(n)$. in line 6, we compute inverse FFT cost $O(n \log n)$ and finally extract first n elements cost $O(n)$. we then add them together will find the total cost is $O(n \log n)$ as wanted.

7 Q7

we solve this problem by using DP. we can firstly separate the final question into small pieces. we define $dp[i]$ as the minimum total cost to i stair.

then recursive formula $dp[i] = \min\{dp[i-1] + c_{i-1,1}, dp[i-2] + c_{i-2,2}\}$, for $i \geq 3$ and $dp[1] = 0$ and $dp[2] = c_{1,1}$

then we consider the base case. $dp[1] = 0$ since we start at first stair and cost nothing and $dp[2] = c_{1,1}$ since move to stair 2 we only have one option. base case hold.

the we illustrate why this formula is correct. For every $i \geq 3$, we have two possible choices:

1. From i-1 using a 1-step move: $dp[i] = dp[i-1] + c_{i-1,1}$
2. From i-2 using a 2-step move: $dp[i] = dp[i-2] + c_{i-2,2}$

then we need to find the minimum of cost then get the formula

what we want to do is to compute the $dp[n]$

then we do our algo, with input are list of costs c where $c_{i,1}$ and $c_{i,2}$ represent the cost to move from stair i and number of stairs n. and output is minimum cost to reach the top (stair n)

Algorithm 1 get(c, n)

```

1:  $prev2 = 0$  // Represents  $dp[i-2]$ 
2:  $prev1 = c_{1,1}$  // Represents  $dp[i-1]$ 
3: for  $i = 3$  to  $n$  do
4:    $current = \min(prev1 + c_{i-1,1}, prev2 + c_{i-2,2})$ 
5:    $prev2 = prev1$ 
6:    $prev1 = current$ 
7: end for
8: Return  $prev1$ 

```

Complexity : since line 1 & 2 cost $O(1)$ and line 3 cost $O(n)$ then total cost is $O(n)$

8 Q8

we using DP to solve this problem. first we define three subproblem value: For $1 \leq i \leq n$

1. $dp[i][0]$ = the maximum profit when reach i plot and grow nothing
2. $dp[i][1]$ = the maximum profit when reach i plot and grow corn
3. $dp[i][2]$ = the maximum profit when reach i plot and grow pumpkins

recursive formula for each subproblem:

1. $dp[i][0] = \max(dp[i-1][0], dp[i-1][1], dp[i-1][2])$ for $1 < i \leq n$ and $dp[1][0] = 0$
 2. $dp[i][1] = \max(dp[i-1][0], dp[i-1][2]) + c_i$ for $1 < i \leq n$ and $dp[1][1] = c_1$
 3. $dp[i][2] = \max(dp[i-1][0], dp[i-1][1]) + p_i$ for $1 < i \leq n$ and $dp[1][2] = p_1$
- what we want in final is $\max(dp[n][0], dp[n][1], dp[n][2])$

Justify why it is correct: for the base case, when we talk about the first plot, we have three choice to grow. The initial values are correctly set: $dp[1][0] = 0$, $dp[1][1] = c_1$, $dp[1][2] = p_1$

IH: assume that for any i, $dp[i][0]$, $dp[i][1]$, $dp[i][2]$ correctly get the maximum up to plot i. WTP: in $i+1$ plot we also get what we want.

$$dp[i+1][0] = \max(dp[i][0], dp[i][1], dp[i][2])$$

$$dp[i+1][1] = \max(dp[i][0], dp[i][2]) + c_{i+1}$$

$$dp[i+1][2] = \max(dp[i][0], dp[i][1]) + p_{i+1}$$

thus, the algorithm correctly computes the optimal profit for all n .

assume c is the profit list of corn and p is the profit list of pumpkins and n is the num of plots

Algorithm 2 Max(c, p, n)

```

1: prev_empty = 0
2: prev_corn =  $c_1$ 
3: prev_pumpkin =  $p_1$ 
4: for  $i = 2$  to  $n$  do
5:   curr_empty =  $\max(\text{prev\_empty}, \text{prev\_corn}, \text{prev\_pumpkin})$ 
6:   curr_corn =  $\max(\text{prev\_empty}, \text{prev\_pumpkin}) + c[i]$ 
7:   curr_pumpkin =  $\max(\text{prev\_empty}, \text{prev\_corn}) + p[i]$ 
8:   prev_empty = curr_empty
9:   prev_corn = curr_corn
10:  prev_pumpkin = curr_pumpkin
11: end for
12: Return  $\max(\text{prev\_empty}, \text{prev\_corn}, \text{prev\_pumpkin})$ 

```

running time: for line 1 – 3 cost $O(1)$ and line 4 cost $O(n)$, thus total is $O(n)$

9 Q9

In this question we need to transform to the question about using Kadane's Algorithm. the question need us to find a sequesnce which should have maximum discrepancy.

in this case, we can transform the string "a", "b", "c" to number "-1", "1", "0". in this way, what we do is as same as Kadane's Algorithm. we need to find the **Maximum Subarray Sum**.

Algorithm 3 Max(x)

```
1: s = []
2: for i = 1 to n do
3:   if x[i] == 'b' then
4:     s[i] = 1
5:   else if x[i] == 'a' then
6:     s[i] = -1
7:   else
8:     s[i] = 0
9:   end if
10: end for
11: max_sum = 0
12: current_sum = 0
13: for i = 1 to n do
14:   current_sum = max(s[i], current_sum + s[i])
15:   max_sum = max(max_sum, current_sum)
16: end for
17: Return max_sum
```

Running time: for line 1-10 we do string to numeric array conversion will cost $O(n)$ and then apply Kadane's Algorithm in line 11-16 will also cost $O(n)$. finally, the total cost is $O(n)$ which is less than $O(n^2)$ as wanted.

correctness: **base case:** If $n = 1$, the only possible substring is x_1 , and the discrepancy is directly computed.

IH: Assume for an array of size i , the algorithm correctly computes the maximum discrepancy. Kadane's algorithm correctly maintains a running maximum subarray sum, which is the optimal solution for any given segment.

Thus, by mathematical induction, the algorithm is correct.

10 Q10

for this question we will use DP to solve this problem. firstly we define the subproblem value:

$dp[i][r] = \text{True}$ if we can achieve remainder r using the first i numbers

recursive step:

$$dp[1][x_1 \bmod d] = \text{True}$$

if we assume a remainder r is reachable at step $i-1$, then we have

$$dp[i][(r + x_i) \bmod d] = \text{True}$$

$$dp[i][(r - x_i + d) \bmod d] = \text{True}$$

finally we decide if $dp[n][k] = \text{True}$. if it is true then return yes, otherwise return no

why it is correct workprove by induction

base case: When $i = 1$, the only possible remainder is $x_1 \bmod d$, which is correctly initialized.

IH. Assume that for $i-1$, all possible mod values are correctly stored in the DP table. For i , we correctly update the DP table using "+" and "-". This ensures all reachable values from previous steps are preserved and extended. Since we correctly track all possible values and check if k is reachable, the algorithm is mathematically correct.

input : x is a list of numbers in $\mathbb{Z}_d$, n is num of elements, d is modulo value and k is the target value

Algorithm 4 Do(x, n, d, k)

```

1: for  $r = 0$  to  $d - 1$  do
2:    $dp[1][r] \leftarrow \text{False}$ 
3: end for
4:  $dp[1][x[1] \bmod d] = \text{True}$ 
5: for  $i = 2$  to  $n$  do
6:   for  $r = 0$  to  $d - 1$  do
7:     if  $dp[i-1][r] == \text{True}$  then
8:        $dp[i][(r + x[i]) \bmod d] = \text{True}$ 
9:        $dp[i][(r - x[i] + d) \bmod d] = \text{True}$ 
10:    end if
11:   end for
12: end for
13: if  $dp[n-1][k] == \text{True}$  then
14:   Return "yes"
15: else
16:   Return "no"
17: end if

```

Running time: in line 1-3, we cost $O(d)$. and in line 5-12, we cost total $O(n \cdot d)$ since the big loop is n times and inner loop is d times. the other is $O(1)$ which is constant time. thus, we know that the total time is $O(d \cdot n)$